



PROMETHEUS

Powerful
Metrics
Collection



ALERTMANAGER

Smart Alerts
& Routing



GRAFANA

Beautiful
Dashboards



WHAT YOU'LL LEARN

- ★ Build scalable monitoring systems
- ★ Design for reliability & HA
- ★ Store metrics long-term
- ★ Write powerful queries
- ★ Alert with confidence
- ★ Troubleshoot like a pro



ADVANCED
PROMQL



ALERTS &
ALERTMANAGER



REMOTE WRITE
& FEDERATION



THANOS
CORTEX & MIMIR



HIGH
AVAILABILITY



TROUBLESHOOTING
& BEST PRACTICES



MASTER MONITORING.
DELIVER RELIABILITY.

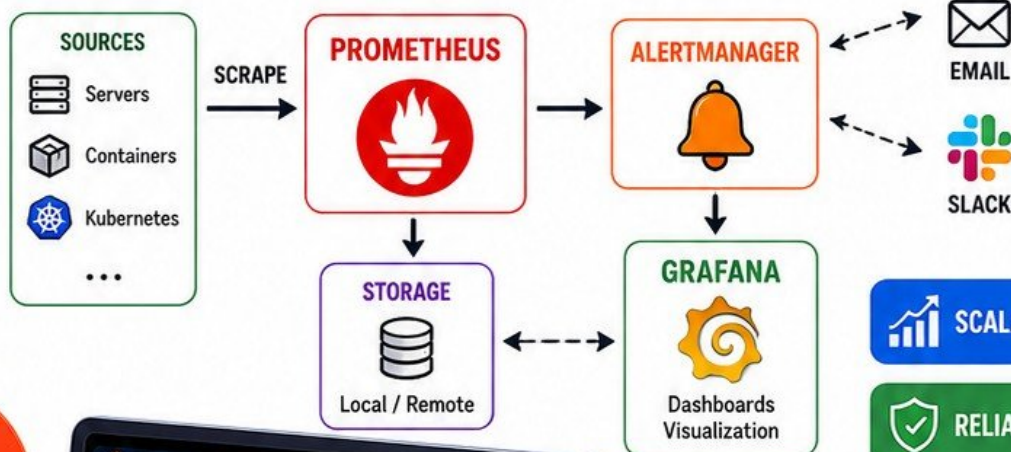
20 PAGES
OF HANDWRITTEN
ENGINEERING NOTES

VERIQTA

PROMETHEUS & GRAFANA HANDBOOK

VOLUME 2

FROM FUNDAMENTALS TO PRODUCTION-SCALE MONITORING



SCALABLE

RELIABLE

OBSERVABLE

PRODUCTION
READY

1. PROMQL FUNDAMENTALS

1 WHAT IS PROMQL?



PromQL (Prometheus Query Language) is the query language used to select, filter, and analyze metrics stored in Prometheus.

It lets you ask questions about your systems and get meaningful answers as numbers, graphs, or time series.

2 WHY PROMQL EXISTS

- ✓ Metrics are raw and high volume.
- ✓ You need a powerful language to extract insights.
- ✓ PromQL helps you:
 - Filter and narrow down data
 - Calculate rates and trends
 - Aggregate across dimensions
 - Build alerts and dashboards



KEY TAKEAWAY

PromQL is how you turn raw metrics into actionable observability.

Know PromQL,
Know Your System.

3 INSTANT VECTOR vs RANGE VECTOR

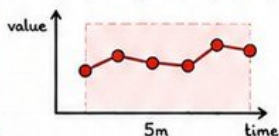
INSTANT VECTOR

- Returns a single data point for each time series at a specific moment.
- Used for current values.
- Example: `cpu_usage`



RANGE VECTOR

- Returns a set of data points over a time range.
- Used for rates, increases, and trends.
- Example: `cpu_usage[5m]`



4 SELECTING METRICS

- Metrics are identified by metric names.
- You can filter using labels (`key=value`).
- Use curly braces `{}` for label matchers.

SYNTAX

```
metric_name{label_name="value", label!="value"}
```

EXAMPLES

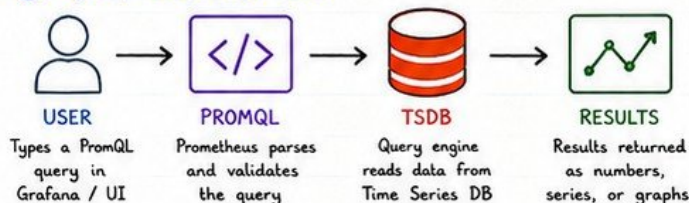
```
node_cpu_seconds_total{mode="idle"}
http_requests_total{method="GET", status="200"}
up{job="node-exporter"}
```



5 BASIC QUERY EXAMPLES

QUERY	WHAT IT DOES	EXAMPLE OUTPUT
<code>up</code>	Check if targets are up (1 = up, 0 = down)	1
<code>node_cpu_seconds_total</code>	Total CPU seconds counter	123456.78
<code>rate(node_cpu_seconds_total[5m])</code>	CPU usage rate over last 5 minutes	0.15
<code>sum by (instance) (up)</code>	Sum up status by instance	1 per instance
<code>avg(rate(http_requests_total[5m]))</code>	Average request rate across all instances	25.4

6 QUERY EXECUTION FLOW



QUERY ENGINE STEPS

- 1 Parse & validate query
- 2 Find matching time series
- 3 Read data from TSDB
- 4 Apply functions & operators
- 5 Return final result



7 COMMON BEGINNER MISTAKES

✗ Wrong time range

Using instant vector when you need range vector (e.g., `rate()` needs `[5m]`)



✗ High cardinality queries

Querying across high-cardinality labels (e.g., `user_id`) can slow down Prometheus.



✗ Using raw counters for trends

Counters only go up. Use `rate()`, `increase()`, not raw values.



✗ Forgetting label filters

Pulling too much data and getting noisy results.



✗ No aggregation

Too many series returned; hard to read and expensive to query.



✗ Ignoring query performance

Complex queries can impact Prometheus and cause timeouts.

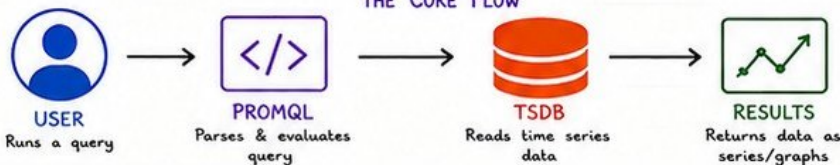


PRO TIP

- Start simple.
- Build step by step.
- Use range vectors for rates and trends.
- Test in Prometheus expression browser.
- Optimize with recording rules when needed.



THE CORE FLOW



PromQL is the bridge between raw metrics and real insights. Master it, and you master observability.

2. ADVANCED PROMQL

★ PromQL gives you powerful functions and operators to extract deep insights from your metrics.



REMEMBER

Everything in PromQL returns a **vector** instant or range.

A. AGGREGATION OPERATORS

sum() avg() max() min() count()

Aggregate across dimensions (labels).

Syntax: <operator>(<vector>) by (label)



B. rate()

Per-second average rate of increase of a counter (over a range vector).
Use for normal alerting & dashboards.



C. irate()

Instantaneous rate based on the last two data points. Shows short-term spikes.
Use for fast-changing metrics.



D. increase()

Total increase of a counter over a time range. Great for reports & billing.



E. sum()

Returns the sum of all values in the vector.



F. avg()

Returns the average of all values in the vector.



G. max()

Returns the highest value in the vector.



H. min()

Returns the lowest value in the vector.



I. topk(k, vector)

Returns the top K time series with the highest values.



J. histogram_quantile(φ, vector)

Calculates quantiles (e.g., 0.95, 0.99) from histogram buckets.

K. REAL PRODUCTION QUERY EXAMPLES

USE CASE	QUERY	EXAMPLE OUTPUT
CPU Usage (rate over 5m)	100 * avg(rate(node_cpu_seconds_total {mode!="idle"}[5m])) by (instance)	42.3 %
HTTP Request Rate (per sec)	sum(rate(http_requests_total[5m])) by (job, instance)	128.7 req/s
Error Rate (5xx %)	100 * sum(rate(http_requests_total {status=~"5.."}[5m])) by (job) / sum(rate(http_requests_total[5m])) by (job)	2.35 %
Total Requests (increase over 1h)	increase(http_requests_total[1h]) by (job)	12,456
Top 5 CPU Consumers	topk(5, 100 * avg(rate(node_cpu_seconds_total {mode!="idle"}[5m])) by (instance))	1) server-01 82% 2) server-02 61% 3) server-03 48% 4) server-04 37% 5) server-05 21%
Average Memory Usage	avg(node_memory_MemAvailable_bytes) by (instance)	3.46 GiB
Disk Usage %	100 - (avg(node_filesystem_avail_bytes) by (instance, mountpoint) / avg(node_filesystem_size_bytes) by (instance, mountpoint)) * 100	67.8 %
Latency p95 (HTTP)	histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le, job))	0.842 s
Pod Restarts (increase 6h)	increase(kube_pod_container_status_restarts_total[6h]) by (namespace, pod)	23
High Card Awareness (count series)	count(count by (___name___, instance)({job="node"}))	1,284 series

PRO TIPS



✓ Use rate() for alerts & long-term trends.

✓ Use irate() for rapid changes & spike detection.

✓ Use by(...) and without(...) to control grouping.

✓ Test queries in Prometheus UI before using in dashboards or alerts.



COMMON PITFALLS

- Using rate() on a gauge metric.
- Forgetting range vector [5m].
- Not aggregating → too many series.
- Ignoring label filters → slow queries.
- Using irate() for long-term dashboards.

✗ Avoid these for production stability!



PromQL is your superpower.
Practice daily,
query everything!



3. RECORDING RULES

Pre-compute and store frequently used PromQL queries as new time series.
Faster dashboards, lower CPU, better scalability.



Do the heavy lifting once.
Re-use the result
everywhere.



1 WHY RECORDING RULES EXIST



- PromQL queries can be expensive (high CPU, memory).
- Complex queries repeated across dashboards & alerts.
- Dynamic calculations over large time ranges are slow.
- Recording rules pre-compute results and store them as new time series.

2 PERFORMANCE BENEFITS



Faster
Dashboards
& Alerts



Lower CPU
& Memory
Usage



Reduced
Query Load
on TSDB



Consistent
Results



Scales Better
in Production

3 YAML CONFIGURATION

groups:

- name: node.rules

interval: 30s

rules:

- record: node:cpu_usage:rate5m

expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

labels:

severity: info

- record: node:memory_usage_bytes

expr: node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes

labels:

severity: info



Explanation

- The query result is stored every interval (30s) as a new time series with the name after 'record'.
- You can then query: node:cpu_usage:rate5m

4 RECORDING RULE WORKFLOW



1. Define Query
(Recording Rule)



2. Prometheus
Evaluates on
interval



3. Result Stored
as New Time
Series



4. Used by
Dashboards,
Alerts, Queries

EXAMPLE FLOW

rate(node_cpu_seconds_total[5m]) → Average per instance → record: node:cpu_usage:rate5m

- Now dashboards query the recorded metric (fast) instead of the raw expensive query.



5 BEST PRACTICES



- Keep interval aligned with scrape_interval (e.g., 30s/60s).
- Record results used in dashboards & alerts frequently.
- Choose meaningful metric names with clear prefixes.
- Use labels to add context (team, service, env, severity).
- Document your rules and group related rules together.
- Test queries in Prometheus UI before adding as rules.
- Monitor rule performance (prometheus_rule_evaluation_duration_seconds).

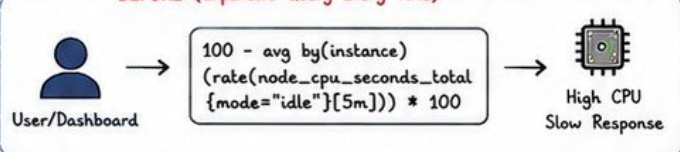
6 COMMON MISTAKES

- Recording everything (creates unnecessary data).
- Using too short interval (increases load & storage).
- Poor naming (confusing & hard to understand).
- Forgetting labels (loses useful context).
- Creating rules for one-time or rarely used queries.
- Chaining too many recording rules (hard to debug).
- Not monitoring rule evaluation duration or failures.

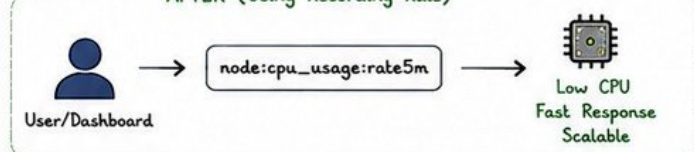


7 BEFORE VS AFTER

BEFORE (Expensive Query Every Time)



AFTER (Using Recording Rule)



Recording rules = Better performance, lower cost, happier dashboards!



4. ALERT RULES



KEY IDEA

Good alerts inform, not annoy. They help you act fast. ✓

1 WHAT ALERT RULES ARE

- Alert rules define when Prometheus should send an alert to Alertmanager.
- They are based on PromQL expressions evaluated at regular intervals.
- When the condition is true for long enough, the alert becomes **FIRING**.

ALERT RULE = CONDITION + DURATION



PromQL
Expression



For how
long?



Notify via
Alertmanager

2 ALERT LIFECYCLE



Inactive
No alert
condition
met.



Pending
Condition is
true, waiting
for duration.



Firing
Condition true
for longer than
duration.



Resolved
Condition is
false again.
Alert clears.

3 THRESHOLD DESIGN



- Choose meaningful thresholds based on:
 - Historical data (baseline + anomalies)
 - SLOs / SLAs
 - Business impact
 - Capacity & saturation levels

TIPS

- ✓ Use percent (%) for resources.
- ✓ Use absolute values for business metrics.
- ✓ Avoid static one-size-fits-all thresholds.
- ✓ Revisit & tune thresholds regularly.

GOOD THRESHOLDS:

- ✓ Detect real issues
- ✓ Reduce noise
- ✓ Trigger early
- ✓ Help quick action

4 PENDING VS FIRING

PENDING

- Condition = true
- But not true long enough
- Waiting for "for" duration to be satisfied
- No notification yet

Example:

for: 5m means condition must be true for 5 minutes continuously.



FIRING

- Condition true for longer than "for" duration
- Alert is sent to Alertmanager
- Notifications are delivered

Why it matters?

Avoids alert flapping due to short spikes.

5 WRITING EFFECTIVE ALERTS

- S** Specific - Clear name & purpose
- M** Measurable - Based on reliable metrics
- A** Actionable - Provides context to act
- R** Relevant - Important to your team
- T** Timely - Alerts you early enough

ANATOMY OF A GOOD ALERT



Alert Name

Short, specific, and meaningful



Description

Explain what happened & why



Runbook / Links

Where to go for more info



Labels

team, service, severity, env



Annotations

Summary, dashboard, runbook URL

BEST PRACTICES

- ✓ Use clear & consistent naming (PascalCase).
- ✓ Include labels: team, service, severity, env.
- ✓ Add summary and description.
- ✓ Link dashboards & runbooks.
- ✓ Keep alerts actionable for the on-call.
- ✓ Test alerts before production.



6 PRODUCTION ALERT EXAMPLES

ALERT NAME	PROMQL EXPRESSION	DURATION (for)	DESCRIPTION	SEVERITY
 HighCPUUsage	$100 - (\text{avg by(instance)} (\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100) > 90$	5m	CPU usage > 90% for 5 minutes.	critical
 HighMemoryUsage	$(\text{node_memory_MemAvailable_bytes} / \text{node_memory_MemTotal_bytes}) * 100 < 15$	10m	Available memory < 15% for 10 minutes.	warning
 DiskSpaceLow	$(\text{node_filesystem_avail_bytes} / \text{node_filesystem_size_bytes}\{\text{fstype} \sim \text{"tmpfs"} \text{overlay"}\}) * 100 < 10$	10m	Disk space < 10% for 10 minutes.	critical
 HighHTTPErrorRate	$\frac{\text{sum}(\text{rate}(\text{http_requests_total}\{\text{status} \sim \text{"5.."}\}[5\text{m}]))}{\text{sum}(\text{rate}(\text{http_requests_total}[5\text{m}]))} * 100 > 5$	5m	5xx error rate > 5% for 5 minutes.	warning



Well-designed alerts **save time, reduce noise, and protect your users.**



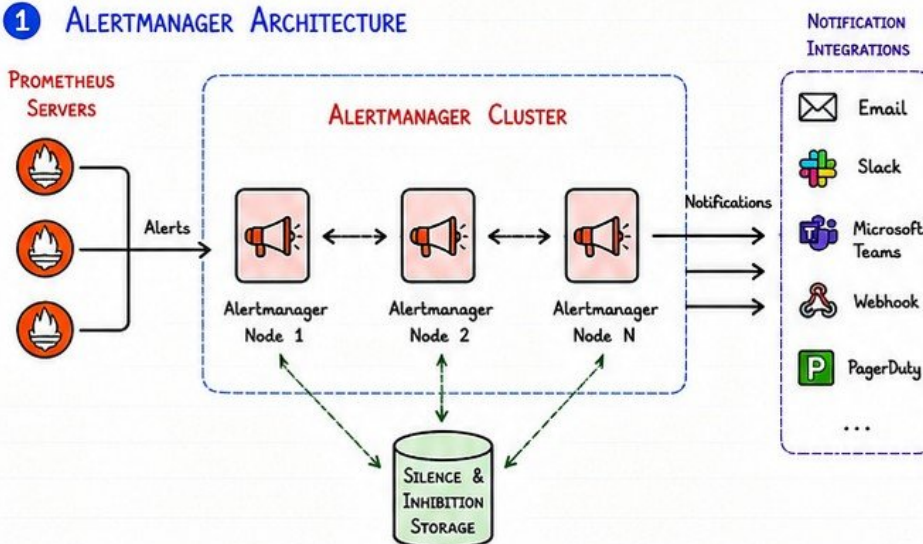
5. ALERTMANAGER

Alertmanager handles alerts from Prometheus. It groups, deduplicates, routes and notifies the right people through the right channel.

KEY IDEA

Alertmanager makes sure the right alert reaches the right person at the right time.

1 ALERTMANAGER ARCHITECTURE



2 ALERT GROUPING

- Alerts with the same label values are grouped together.
- The first alert starts the group.
- Subsequent alerts are added to it.
- A single notification is sent for the group.

GROUP BY EXAMPLE

Group By (Label)	Example Value
alertname	HighCPU
instance	server-01
severity	critical

All alerts with the above labels will be grouped as one.

3 DEDUPLICATION

- Prevents sending the same alert multiple times.
- Based on alert fingerprint (labels + annotations).
- Re-notification happens only after a configurable interval.



Benefit: Reduces noise and alert fatigue.

4 SILENCING

- Silences mute alerts that match specific criteria.
- Useful during maintenance or known issues.
- Silences can be time-bound.

SILENCE MATCHER EXAMPLE

alertname = "HighCPU"
instance = "server-.*"
severity = "warning"

Starts: 2025-07-16 10:00
Ends: 2025-07-16 14:00



5 INHIBITION

- Hides (suppresses) alerts if a more critical alert is firing.
- Example: Don't alert on High CPU if the server is Down.

INHIBITION RULE EXAMPLE

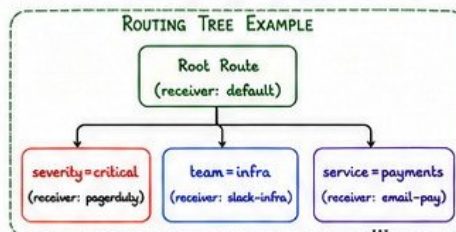
Source (higher severity):
alertname = "ServerDown"
Target (to inhibit):
alertname = "HighCPU"
Equal Labels: instance



6 ROUTING

- Routes alerts to the correct receiver based on label matchers.
- Uses a tree-like routing configuration.
- Supports nested routes and fallback (default receiver).

ROUTING TREE EXAMPLE



ROUTING MATCH EXAMPLE

Incoming Alert Labels	Matched Route → Receiver
severity=critical, team=infra	severity=critical → pagerduty
team=infra	team=infra → slack-infra
service=payments	service=payments → email-pay
(no match)	default → default-receiver

7 NOTIFICATION FLOW



PRO TIPS

- Choose clear group-by labels to reduce noise.
- Use silences during maintenance windows.
- Keep routing rules simple and test often.
- Monitor Alertmanager health and performance.
- Prefer actionable alerts over too many alerts.

— VERIQTA —

6. EMAIL ALERTS



KEY IDEA

Email alerts ensure critical incidents reach the right people, anytime, even when other channels fail.



Email notifications via Alertmanager using SMTP let you deliver reliable, trackable alerts directly to inboxes.

1 SMTP CONFIGURATION

Configure your SMTP server in Alertmanager.

```
# alertmanager.yml
global:
  smtp_smarthost: 'smtp.gmail.com:587'
  smtp_from: 'alerts@veriqta.com'
  smtp_auth_username: 'alerts@veriqta.com'
  smtp_auth_password: 'YOUR_APP_PASSWORD'
  smtp_require_tls: true
  smtp_hello: 'veriqta.com'
```



Notes:

- Use an App Password (Gmail/Outlook)
- Port 587 (STARTTLS) recommended
- smtp_require_tls: true for security

2 ALERT TEMPLATES (EMAIL)

Use templates to format clear, actionable emails.

```
Subject: {{ template "email.subject" . }}

{{ define "email.subject" }}
[[{ .Status | toUpper }}] {{ .CommonLabels.alertname }}
{{ end }}

{{ define "email.body" }}
Alert: {{ .CommonLabels.alertname }}
Status: {{ .Status }}
Severity: {{ .CommonLabels.severity }}
Summary: {{ .CommonAnnotations.summary }}
Description: {{ .CommonAnnotations.description }}
Starts At: {{ .StartsAt }}
Dashboard: {{ .CommonAnnotations.dashboard }}
{{ end }}
```

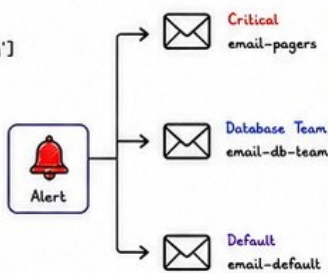
★ BEST PRACTICES

- ✓ Keep subject short and clear
- ✓ Include severity & summary
- ✓ Add runbook / dashboard links
- ✓ Avoid noisy content

3 ROUTING ALERTS

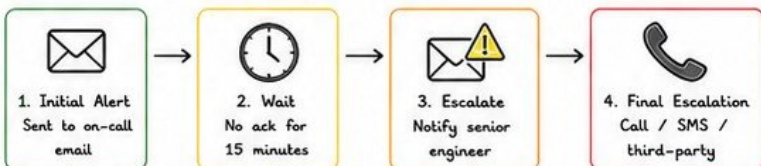
Route alerts to email receivers based on labels.

```
route:
  receiver: 'email-default'
  group_by: ['alertname', 'severity']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 3h
  routes:
    - match:
        severity: critical
      receiver: 'email-pagers'
    - match:
        team: database
      receiver: 'email-db-team'
```



4 ESCALATION STRATEGY

Escalate if no one acknowledges the alert.



```
receivers:
- name: 'email-pagers'
  email_configs:
    - to: 'oncall@veriqta.com'
      send_resolved: true
- name: 'email-escalation'
  email_configs:
    - to: 'senior@veriqta.com'
      send_resolved: true
  headers:
    X-Priority: '1'
```

Use repeat_interval and escalation rules to re-notify until resolved or acknowledged.



5 PRODUCTION EMAIL EXAMPLE

[FIRING] HighCPUUsage Inbox x

From: alerts@veriqta.com
To: oncall@veriqta.com
Time: 2025-07-16 10:45:12 (UTC)

● ALERT: HighCPUUsage
Status: FIRING
Severity: critical
Summary: CPU usage > 90% for 5 minutes on server-01
Description: CPU usage is above 90% for more than 5 minutes. This can impact application performance.
Starts At: 2025-07-16 10:40:00 (UTC)
Instance: server-01
Dashboard: <https://grafana.veriqta.com/d/cpu-overview>
Runbook: <https://runbooks.veriqta.com/high-cpu>

Acknowledge

View Dashboard

6 TROUBLESHOOTING DELIVERY

ISSUE	POSSIBLE CAUSE	HOW TO FIX
✗ Email not received	• Wrong SMTP settings • Firewall blocking • Spam folder	• Verify smtp_smarthost, port, auth • Allow outbound SMTP (587/465) • Check spam / promotions folder
⚠ Authentication failed	• Wrong username/password • App password not used • 2FA enabled	• Use correct username/password • Generate App Password • Ensure 2FA + App Password
🕒 Delayed emails	• High load on SMTP server • Network latency • Alertmanager overloaded	• Use reliable SMTP provider • Check network connectivity • Scale Alertmanager if needed
ℹ TLS / SSL errors	• Invalid TLS config • Certificate issues	• Set smtp_require_tls: true • Verify certificate / port

PRO TIPS



✓ Use a dedicated SMTP account.

✓ Monitor email delivery logs.

✓ Test emails regularly (end-to-end).

✓ Keep templates simple & clear.

✓ Have a fallback channel (Slack / PagerDuty).



7. SLACK ALERTS



KEY IDEA

Good Slack alerts are clear, actionable and routed to the right channel at the right time.



Slack alerts deliver real-time notifications to your team so the right people see the right alerts, instantly.

1 SLACK WEBHOOK SETUP



1. Create a Slack App
Go to api.slack.com/apps
→ Create New App



2. Incoming Webhooks
Enable Incoming Webhooks
for your app.



3. Add to Channel
Choose the channel where
alerts will be posted.



4. Copy Webhook URL
Copy the Webhook URL
(You'll use it in Alertmanager).

alertmanager.yml

```
receivers:  
- name: 'slack-alerts'  
  slack_configs:  
    - api_url: 'https://hooks.slack.com/  
      services/T00000000/B00000000/  
      XXXXXXXXXXXXXXXXXXXXXXXXXXXX'  
      channel: '#alerts-critical'  
      username: 'Alertmanager'  
      icon_emoji: ':rotating_light:'
```

Notes:

- Keep the webhook URL secret.
- Use different webhooks for different channels if needed.

2 NOTIFICATION TEMPLATES

Clear, consistent and actionable messages.

```
{{ define "slack.default.title" }}  
[{{ .Status | toUpper }}] [{{ .CommonLabels.alertname }}]  
{{ end }}  
  
{{ define "slack.default.text" }}  
*Alert:* [{{ .CommonLabels.alertname }}]  
*Status:* [{{ .Status }}]  
*Severity:* [{{ .CommonLabels.severity }}]  
*Summary:* [{{ .CommonAnnotations.summary }}]  
*Description:* [{{ .CommonAnnotations.description }}]  
*Starts At:* [{{ .StartsAt }}]  
*Dashboard:* [{{ .CommonAnnotations.dashboard }}]  
*Runbook:* [{{ .CommonAnnotations.runbook }}]  
{{ end }}
```

PRO TIPS



- ✓ Keep messages short and readable.
- ✓ Include severity, summary & actions.
- ✓ Link dashboards and runbooks.
- ✓ Use emojis to improve visibility.

3 CHANNEL ROUTING

Route alerts to the right channel based on labels.

```
route:  
  receiver: 'slack-default'  
  group_by: ['alertname', 'severity']  
  group_wait: 10s  
  group_interval: 5m  
  repeat_interval: 3h  
  routes:  
  - match:  
    severity: critical  
    receiver: 'slack-critical'  
  - match:  
    severity: warning  
    receiver: 'slack-warning'  
  - match:  
    team: database  
    receiver: 'slack-database'  
  - match_re:  
    service: 'payments|billing'  
    receiver: 'slack-payments'
```

#alerts-critical
(critical issues)



#alerts-warning
(warning issues)



#db-team
(database alerts)



#payments-team
(payments issues)



#alerts-general
(all others)



4 SEVERITY-BASED ALERTS

Different severity, different signal.

Severity	Emoji	Channel	Use Case
critical	🚨 :rotating_light:	#alerts-critical	• Service Down • Data Loss • Security Breach
warning	⚠️ :warning:	#alerts-warning	• High Resource Usage • Threshold Breach • Degraded Performance
info	🔔 :information_source:	#alerts-info	• Deployments • Changes • Informational Events
resolved	✅ :white_check_mark:	#alerts-resolved	• Alert Resolved Notifications



Severity helps reduce noise and improves response time.

5 BEST PRACTICES



- ✓ Send actionable alerts, not just notifications.
- ✓ Include affected service, summary, and runbook.
- ✓ Route alerts to the right team/channel.
- ✓ Use severity and labels consistently.
- ✓ Avoid alert storms (use grouping, inhibitions).
- ✓ Test Slack alerts regularly.
- ✓ Keep messages concise and scannable.

EXAMPLE MESSAGE ELEMENTS

- What happened?
- Where is it happening?
- Impact?
- What should I do?
- Where to investigate?



Goal:

Alert fast,
Inform clearly,
Resolve quickly.



6 REAL INCIDENT EXAMPLE

Production Incident: High CPU on API Servers



Alertmanager APP 10:21 AM

[FIRING] HighCPUUsage

Severity: critical SERVICE: api-gateway ENVIRONMENT: production
Summary: CPU usage > 90% for 5 minutes on 2 instances
Description: CPU usage is above 90% on 2/6 API instances.
This can impact response time and customer experience.
Starts At: 2025-07-16 10:21:00 (UTC)
Dashboard: <https://grafana.veriqta.com/d/api-overview>
Runbook: <https://runbooks.veriqta.com/high-cpu>
Labels: team=platform service=api-gateway severity=critical

View Dashboard

Runbook

Acknowledge



PRO TIPS



Right alert,
right person.



Keep it short,
keep it clear.



Route smart,
reduce noise.



Link dashboards
& runbooks.



Test, monitor
and improve.



Happy Team,
Fast Response!

8. KUBERNETES MONITORING

Kubernetes is dynamic. Things come and go constantly. We combine multiple exporters to get full visibility into **nodes, pods, and workloads**.



KEY IDEA

Use the right tool for the right data to achieve complete observability in Kubernetes.



1 KUBE-STATE-METRICS

- Exposes Kubernetes object metrics via the Kubernetes API.
- Provides metrics about resources state, not resource usage.

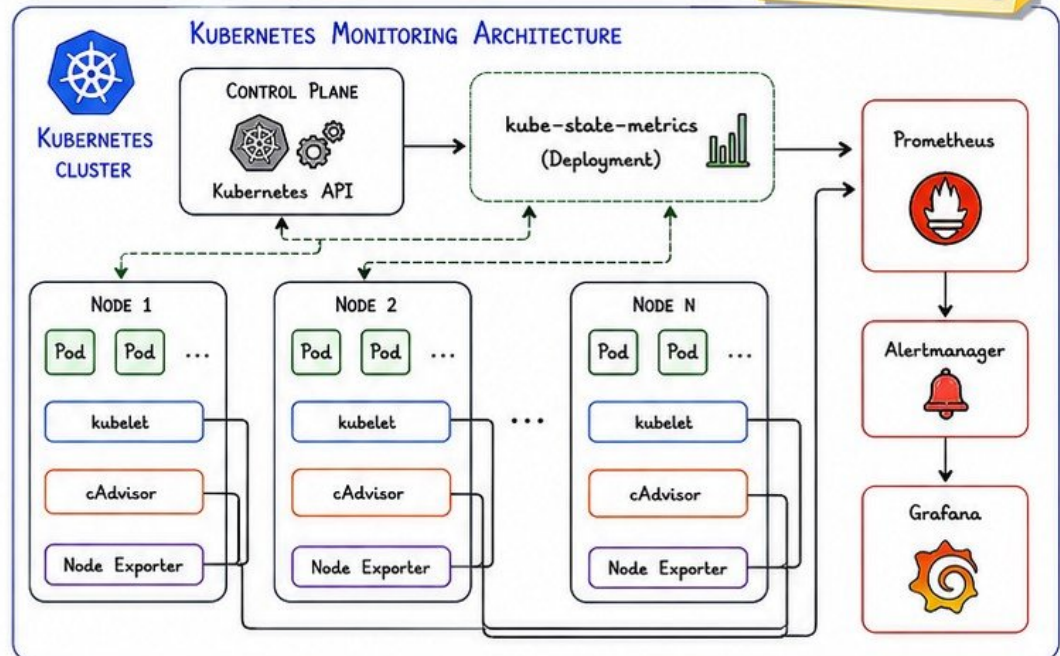
Examples: deployments, replicaset, nodes, pods, services, configmaps, etc.

2 CADVISOR

- Embedded in kubelet.
- Provides container (cgroup) metrics.
- CPU, memory, filesystem, network usage per container.

3 NODE EXPORTER

- Runs on every node.
- Exposes host-level metrics.
- CPU, memory, disk, network, load, filesystem, processes, etc.



4 WHAT DO WE MONITOR?

4.1 NODES



- Node status (ready, not ready)
- CPU, memory, disk usage
- Filesystem usage
- Network I/O
- Node conditions

Tool: Node Exporter

4.2 PODS



- Pod phase (running, pending, failed)
- Restart count
- CPU & memory usage
- Container I/O, network
- Up/Down status

Tool: cAdvisor + kube-state-metrics

4.3 DEPLOYMENTS



- Desired vs available replicas
- Update status (progressing, failed)
- Rollout metrics
- Replicas ready

Tool: kube-state-metrics

4.4 STATEFULSETS



- Replicas status
- Update revision
- Volume claims
- Pod ordinal
- Ready vs desired

Tool: kube-state-metrics

4.5 DAEMONSETS



- Desired vs current scheduled
- Number available
- Node coverage
- Update status

Tool: kube-state-metrics

4.6 SERVICES



- ClusterIP / NodePort / LoadBalancer
- Endpoints up/down
- Request metrics (via app)
- Service availability

Tool: kube-state-metrics

4.7 CONFIGMAPS & SECRETS



- Object count
- Age
- Status
- Change events

Tool: kube-state-metrics

4.8 EVENTS



- Warnings & Errors
- Failed scheduling
- OOMKilled
- Image pull errors

Tool: kube-state-metrics



WHY THIS MATTERS

- ✓ Full visibility across cluster, nodes and workloads.
- ✓ Faster incident detection & root cause analysis.
- ✓ Capacity planning and optimization.
- ✓ Alert before users are impacted.

BEST PRACTICES

- ✓ Run one Node Exporter per node.
- ✓ Deploy kube-state-metrics in HA.
- ✓ Use ServiceMonitor / PodMonitor (Prometheus Operator).
- ✓ Label everything for clean filtering.
- ✓ Keep retention & cardinality under control.



EXAMPLE USE CASE



You see high memory usage on a node.
→ Node Exporter shows host memory.
→ cAdvisor shows container using memory.
→ Pod info shows which deployment owns it.
→ You alert the right team with context.



Right metrics. Right tools. Complete visibility. That's Kubernetes monitoring.



9. NODE MONITORING

Node monitoring helps you understand the health and performance of the underlying infrastructure that runs your workloads.



KEY IDEA

Nodes are the foundation of your cluster. Monitor them closely to prevent slowdowns, outages, and service disruptions. ✓

1 WHAT IS NODE MONITORING?

Monitoring physical or virtual machines (worker & control-plane nodes) to track their resource usage, availability and performance over time.

CORE TOOLS

- Node Exporter (metrics)
- Prometheus (collection)
- Grafana (visualization)
- Alertmanager (alerting)



2 WHAT TO MONITOR ON NODES

2.1 CPU



- CPU usage (%)
- CPU cores
- CPU load
- CPU steal time

2.2 MEMORY



- Memory usage
- Available memory
- Swap usage
- Memory pressure

2.3 DISK



- Disk read/write IOPS
- Disk read/write time
- Disk throughput
- Disk utilization (%)

2.4 FILESYSTEM



- Filesystem usage (%)
- Free space
- Inodes usage
- Mount status

2.5 NETWORK



- Network in/out (bytes)
- Network packets
- Errors / drops
- Interface status

2.6 LOAD AVERAGE



- 1m load average
- 5m load average
- 15m load average
- CPU saturation

2.7 PROCESSES



- Total processes
- Running processes
- Zombie processes
- Top resource hogs

2.8 SYSTEM UPTIME



- Uptime duration
- Reboots count
- Kernel version
- OS information

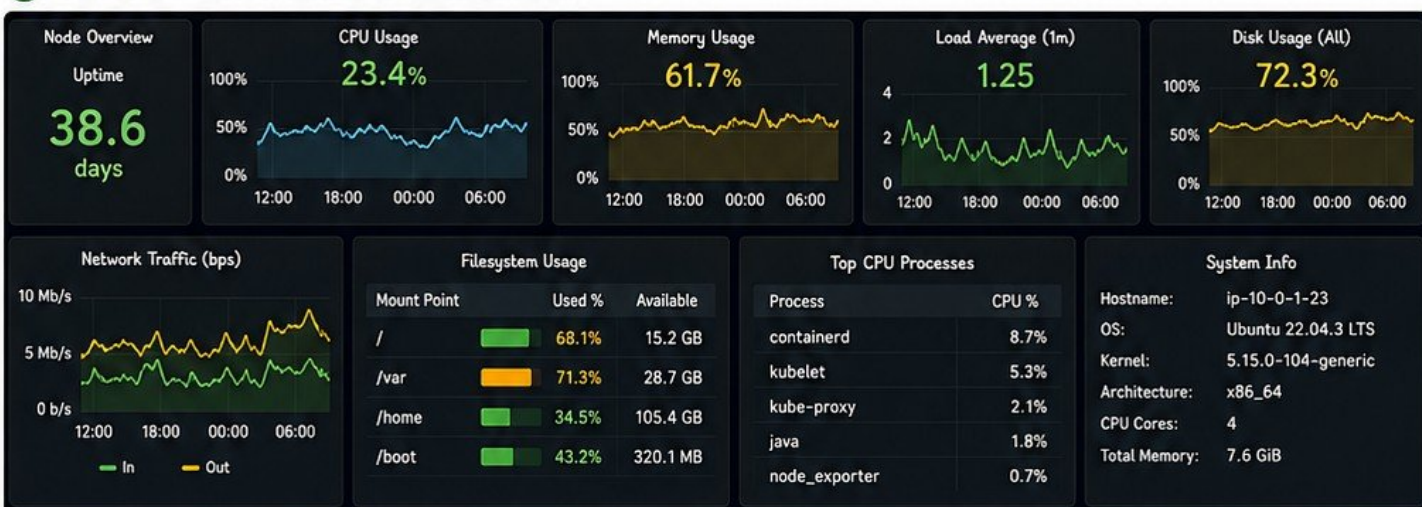
3 NODE EXPORTER METRICS EXAMPLES

METRIC NAME	DESCRIPTION
node_cpu_seconds_total	CPU time spent in each mode
node_memory_MemAvailable_bytes	Available memory in bytes
node_filesystem_avail_bytes	Available filesystem space
node_network_receive_bytes_total	Total bytes received
node_load1	1 minute load average
node_time_seconds	Current time (epoch)
node_boot_time_seconds	System boot time (epoch)
node_uname_info	Kernel, OS, architecture info

4 COMMON ALERTING THRESHOLDS

METRIC	CONDITION (EXAMPLE)	SEVERITY	DESCRIPTION
CPU Usage	> 80% for 5m	critical	Node CPU is highly utilized
Memory Usage	> 85% for 5m	warning	Node memory nearly full
Disk Usage	> 90% for 5m	critical	Disk is almost full
Filesystem Inodes	> 90% for 5m	warning	Inode exhaustion risk
Load Average (1m)	> CPU cores * 2 for 5m	warning	High system load
Node Down	up == 0 for 5m	critical	Node is unreachable

5 PRODUCTION NODE DASHBOARD EXAMPLE (GRAFANA)



PRO TIPS



✓ Monitor everything that can fail.

✓ Set alerts before users are impacted.

✓ Use dashboards for fast troubleshooting.

✓ Baseline normal behavior.

✓ Automate remediation where possible.



10. DOCKER MONITORING

KEY IDEA

Monitor your containers like a pro. Visibility into Docker helps you detect issues early and keep services healthy.



Docker monitoring gives you visibility into container **performance**, **health**, and **resource usage**. It helps you run faster, safer, and more reliable systems.

1 DOCKER METRICS OVERVIEW

Docker exposes rich metrics about:



Containers
Performance, health, resource usage



Images
Pulls, size, layers



Networks
I/O, connections, errors



Volumes
Usage, mounts, status



Daemon
Engine, API, events

2 KEY DOCKER METRICS (VIA cAdvisor + DOCKER EXPORTER)

CONTAINERS

- CPU usage (%)
- Memory usage (RSS)
- Memory limit
- CPU throttling
- Network I/O
- Block I/O
- Restart count
- Uptime



IMAGES

- Total images
- Image size
- Layers count
- Pull count
- Dangling images



NETWORKS

- Network I/O (in/out)
- Packets
- Errors / drops
- Container connections
- Network usage



VOLUMES

- Volume usage
- Mount count
- Read / write bytes
- I/O operations
- Driver status



DOCKER DAEMON

- Container count
- Image count
- Engine version
- Events
- API request rate



SYSTEM (cAdvisor)

- Per-container CPU
- Per-container Memory
- Filesystem usage
- Network usage
- Process count



3 cAdvisor

cAdvisor (Container Advisor) runs on each Docker host and collects container resource usage and performance metrics.

WHAT IT COLLECTS

- CPU
- Memory
- Filesystem
- Network
- Process
- Per container



HOW IT WORKS

1. Runs as a container
2. Talks to Docker Engine
3. Collects metrics
4. Exposes on /metrics
5. Scraped by Prometheus



4 CONTAINER LIFECYCLE MONITORING

Track every stage of a container.



Monitor Events:

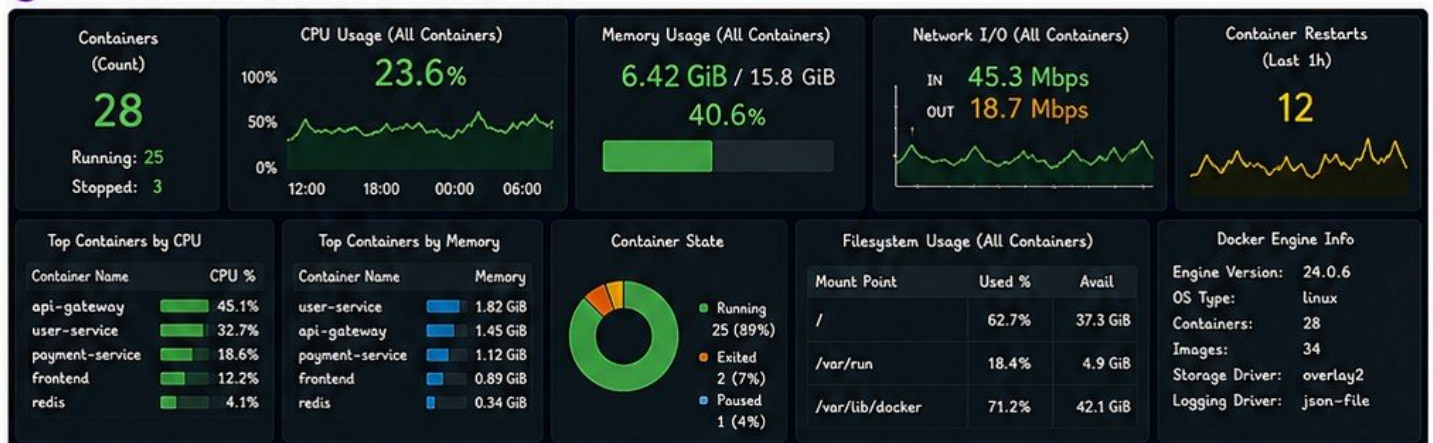
- Container created / started / stopped / removed
- OOMKilled
- Restart (manual / policy)
- Health status changes
- Exit codes
- Errors / failures

5 BEST PRACTICES

- ✓ Monitor all containers, not just critical ones.
- ✓ Set alerts for restarts, OOMKills, and high usage.
- ✓ Use cAdvisor for per-container metrics.
- ✓ Use Docker Exporter for engine-level metrics.
- ✓ Track resource limits vs usage.
- ✓ Monitor logs side-by-side with metrics.
- ✓ Visualize with Grafana dashboards.



6 PRODUCTION DOCKER DASHBOARD EXAMPLE (GRAFANA)



PRO TIPS



Set alerts for restarts & OOMKills.



Track CPU/Memory limits vs usage.



Monitor I/O for noisy neighbors.



Keep an eye on engine events.



Dashboards + Alerts = Fast incident response.



11. APPLICATION METRICS

Application metrics tell you how your software is performing from the **user's** perspective. They help you **detect issues** early, improve performance, and drive **business outcomes**.

KEY IDEA
Great applications expose the right metrics. Monitor what matters to users and your business.

1 WHY INSTRUMENT APPLICATIONS?

- ✓ Know how users experience your app
- ✓ Detect problems before users complain
- ✓ Measure performance and reliability
- ✓ Understand the impact of changes
- ✓ Make data-driven engineering decisions
- ✓ Track business outcomes

HOW TO INSTRUMENT

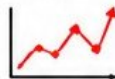
- Expose metrics endpoint (e.g. /metrics)
- Use client libraries (Prometheus client)
- Add meaningful labels (method, status, route)
- Keep labels low-cardinality
- Test your metrics!



2 RED METHOD (FOR REQUEST-BASED SYSTEMS)

A simple way to monitor application performance.

RATE



How many requests are happening?

Example: `requests_total`

ERROR



How many requests are failing?

Example: `errors_total`

DURATION



How long do requests take?

Example: `request_duration_seconds`

Use RED together for full visibility into your system.

3 GOLDEN SIGNALS (FOR MODERN SYSTEMS)

Defined by Google SRE. Works for any service.

1. LATENCY



How long does it take to serve a request?

(e.g. p50, p95, p99)

2. TRAFFIC



How much demand is placed on the system?

(e.g. requests/sec)

3. ERRORS



How often are requests failing?

(e.g. error rate %)

4. SATURATION



How full is the system / its resources?

(e.g. CPU, queue size)

5. AVAILABILITY



Is the system available when needed?

(e.g. uptime %)

Together, they show the health of your service end-to-end.

4 HTTP METRICS

Track all incoming HTTP requests.

```
http_requests_total{method="GET",  
  route="/api/users", status="200"} 1254  
http_requests_total{method="POST",  
  route="/api/users", status="201"} 210  
http_request_duration_seconds_bucket{  
  le="0.1", method="GET", route="/api/users"} 320  
http_request_duration_seconds_sum{...} 45.2  
http_request_duration_seconds_count{...} 1254
```

Track by:

- Method (GET, POST)
- Route/Endpoint
- Status Code (2xx, 4xx, 5xx)
- Service / Version

Useful KPIs

- ✓ Request Rate (QPS)
- ✓ Success Rate (%)
- ✓ P95 / P99 Latency

5 ERROR METRICS

Measure failures explicitly.

EXAMPLES

- 5xx responses
- Exceptions thrown
- Validation errors
- Timeouts
- Business logic errors

EXAMPLE METRIC

```
app_errors_total{type="timeout",  
  route="/api/payment"} 37
```

Alert on error rates, not just counts!



6 LATENCY METRICS

Measure how long requests take.

USE HISTOGRAMS



- Capture latency distribution (not just average)
- Track p50, p90, p95, p99
- Set SLOs on p95/p99

EXAMPLE QUERY

```
histogram_quantile(0.95,  
  sum(rate(http_request_duration_  
    seconds_bucket[5m])) by (le, route))
```



Focus on high percentiles (p95, p99) for user experience.

7 BUSINESS METRICS

Connect technical metrics to business outcomes.

EXAMPLES

- Orders created
- Payments processed
- User signups
- Active users
- Revenue generated

EXAMPLE METRIC

```
app_orders_created_total(status="success") 842
```

Business metrics show impact, not just activity.



8 BEST PRACTICES

- ✓ Expose a /metrics endpoint for your app.
- ✓ Use meaningful, low-cardinality labels.
- ✓ Follow naming conventions (service_metric_target_unit).
- ✓ Prefer counters & histograms.
- ✓ Document your metrics.
- ✓ Alert on symptoms, not root causes (e.g., latency, errors).
- ✓ Review metrics regularly and remove unused ones.



PRO TIPS



RED + Golden Signals = Complete visibility.



High latency + low errors = Still bad UX.



Instrument business flows, not just APIs.



Test metrics in staging before production.



Metrics without alerts are just expensive graphs.

12. CUSTOM METRICS

Built-in metrics are great, but your application is unique.

Custom metrics let you measure **what really matters** to your users and business.



KEY IDEA

Built-in metrics give you infrastructure visibility.
Custom metrics give you business and application visibility.



1 WHEN BUILT-IN METRICS AREN'T ENOUGH

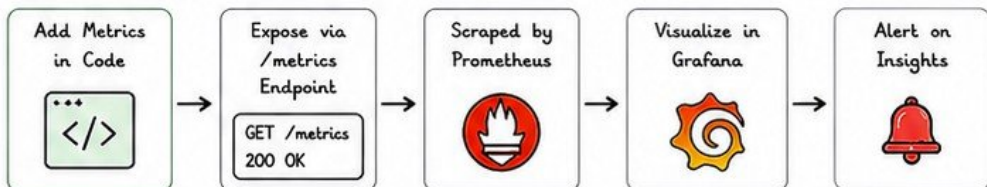
- ✓ Business KPIs (orders, revenue, signups)
- ✓ Application logic (cache hits, queue size)
- ✓ Domain events (payments, approvals)
- ✓ Feature usage (new feature adoption)
- ✓ SLAs and error budgets
- ✓ Deep performance insight



EXAMPLE SCENARIO

You see high CPU, but why?
Is it user traffic? Failed payments? Slow queries?
Custom metrics tell the story.

2 INSTRUMENTING APPLICATIONS



Example (/metrics output)

```
http_requests_total{method="GET",status="200"} 12543
http_requests_total{method="POST",status="500"} 37
app_orders_created_total 842
app_payment_failed_total 19
app_active_users 1523
app_queue_depth 27
```

Client Libraries

- Python: prometheus_client
- Java: Micrometer
- Go: client_golang
- Node.js: prom-client
- .NET: prometheus-net

3 PROMETHEUS METRIC TYPES

COUNTER



Monotonically increasing value that only goes up.

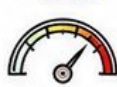
Use for:

- Requests total
- Errors total
- Orders created
- Payments processed

Example

```
http_requests_total 12543
orders_created_total 842
```

GAUGE



Value can go up or down.

Use for:

- Current temperature
- Active users
- Queue depth
- Memory usage

Example

```
app_active_users 1523
queue_depth 27
```

HISTOGRAM



Tracks the distribution of values across configurable buckets.

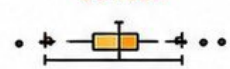
Use for:

- Request duration
- Response size
- Query execution time

Example (buckets)

```
http_request_duration_seconds_bucket{le="0.1"} 1200
http_request_duration_seconds_bucket{le="0.5"} 3400
http_request_duration_seconds_bucket{le="1"} 5200
http_request_duration_seconds_count 5600
http_request_duration_seconds_sum 2780.4
```

SUMMARY



Similar to histogram but calculates quantiles (e.g., p50, p95, p99).

Use for:

- Latency with quantiles
- Small number of time series
- Client-side quantiles

Example

```
http_request_duration_seconds{quantile="0.5"} 0.120
http_request_duration_seconds{quantile="0.95"} 0.480
http_request_duration_seconds_count 5600
http_request_duration_seconds_sum 2780.4
```



Tip: Prefer Histogram for server-side monitoring (scalable, aggregatable). Use Summary for client-side or small-scale needs.

4 BEST PRACTICES

- ✓ Name metrics with a consistent prefix (e.g., app_).
- ✓ Use snake_case for metric names.
- ✓ Use meaningful labels (method, status, region).
- ✓ Keep cardinality low (avoid high-cardinality labels like user_id).
- ✓ Use proper metric types (Counter vs Gauge).
- ✓ Document your metrics (what, why, labels).
- ✓ Alert on metrics that matter to users, not just systems.
- ✓ Review metrics regularly and remove unused ones.

CARDINALITY GUIDE

Good: status, method, region

Bad: user_id, session_id, request_id



5 FROM BUSINESS QUESTION TO METRIC

BUSINESS QUESTION	EXAMPLE METRIC	TYPE
How many orders are we getting?	app_orders_created_total	Counter
How many active users right now?	app_active_users	Gauge
How fast are requests?	http_request_duration_seconds	Histogram
How many payments failed?	app_payment_failed_total	Counter
Is the queue backed up?	app_queue_depth	Gauge
What is our success rate?	app_requests_total + app_errors_total	Counter

PRO TIPS



Measure what matters.



Use labels wisely.



Prefer histograms for latency.



Document and version metrics.



Remove what you don't use.



Alert on user impact, not noise.

13. PUSHGATEWAY

Pushgateway is a Prometheus companion service that accepts metrics from **short-lived jobs** and exposes them for scraping.



KEY IDEA

Pushgateway lets short-lived jobs expose metrics to Prometheus by pushing them, since they can't be scraped.

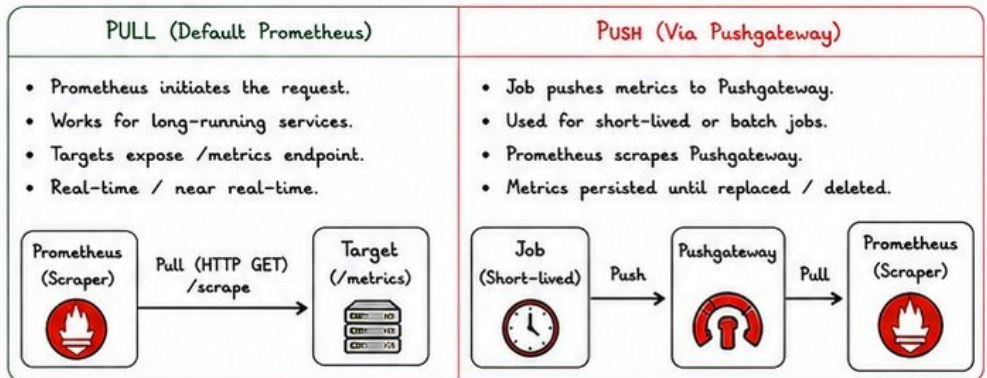


1 WHY PUSHGATEWAY EXISTS

- ✓ Prometheus pulls (scrapes) metrics.
- ✓ Batch / short-lived jobs exit before they can be scraped.
- ✓ Pushgateway acts as a temporary buffer / endpoint.
- ✓ Prometheus scrapes Pushgateway and gets the metrics.



2 PUSH VS PULL

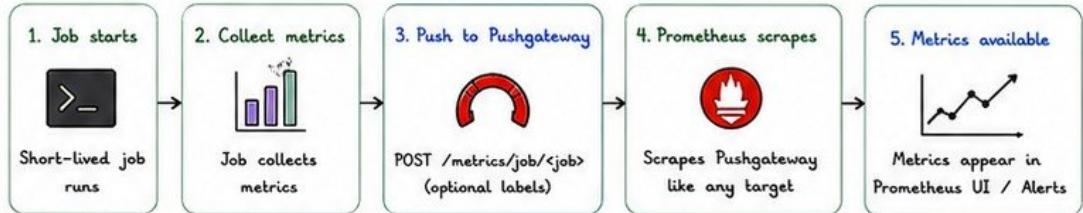


3 USE CASES

- ✓ Batch jobs
- ✓ Cron jobs
- ✓ Data migrations
- ✓ Backup / restore jobs
- ✓ One-time scripts



4 HOW IT WORKS (HIGH LEVEL)



5 PUSHGATEWAY API EXAMPLES

```
# Push metrics for a job
curl -X POST http://pushgateway:9091/metrics/job/daily-report \
-d 'report_status 1
report_duration_seconds 42.5'

# Push with instance label
curl -X POST http://pushgateway:9091/metrics/job/daily-report/instance/server-1 \
-d 'report_status 1'

# Delete metrics for a job
curl -X DELETE http://pushgateway:9091/metrics/job/daily-report

# Delete metrics for a job + instance
curl -X DELETE http://pushgateway:9091/metrics/job/daily-report/instance/server-1
```

6 LIMITATIONS

- ✗ Not for long-running services.
- ✗ Stored metrics can become stale.
- ✗ No built-in TTL for metrics.
- ✗ Overuse leads to high cardinality.
- ✗ Extra component to operate.

IMPORTANT

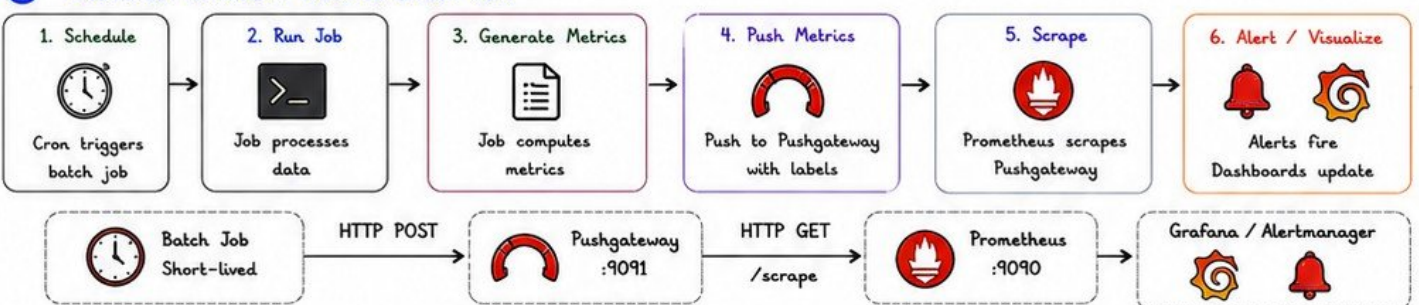
Always delete metrics after job completion to avoid stale data!



7 COMMON MISTAKES

- ✗ Using Pushgateway for services instead of jobs.
- ✗ Not deleting old metrics.
- ✗ Pushing high-cardinality labels (user_id, session_id, etc.).
- ✗ Not monitoring Pushgateway itself.
- ✗ Forgetting to include meaningful labels (job, instance, env).

8 PRODUCTION WORKFLOW EXAMPLE (BATCH JOB)



PRO TIPS



Always delete metrics after job completes.



Use meaningful labels: job, instance, env.



Monitor Pushgateway memory & size.



Restrict Pushgateway access (network / auth).



Keep metrics fresh, not stale!

— VERIQTA —

14. FEDERATION



KEY IDEA

Federation lets you scale Prometheus by scraping metrics from other Prometheus servers. Monitor big systems without one giant server.



Federation allows you to collect metrics from multiple Prometheus servers in a **hierarchical** way. It scales monitoring and gives you a global view.

1 WHY FEDERATION EXISTS

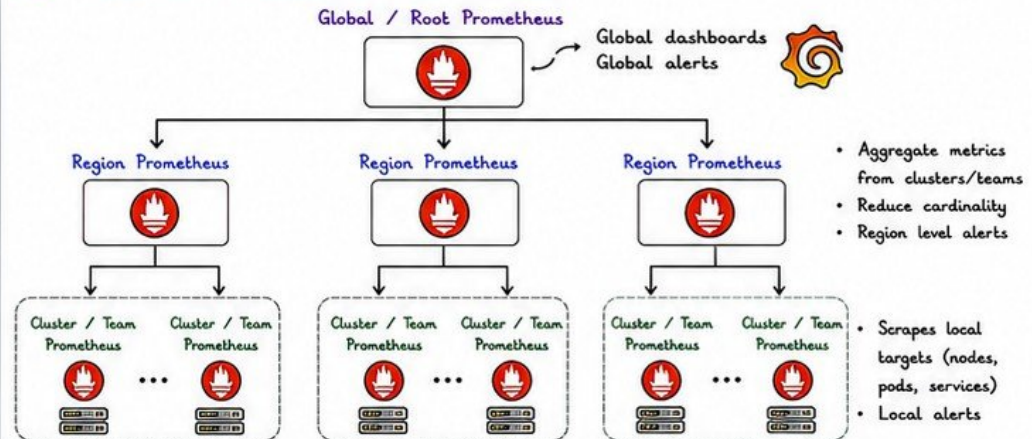
- ✓ Single Prometheus can't handle unlimited scrape targets.
- ✓ High cardinality → performance issues.
- ✓ Need global view across many clusters, regions, or teams.
- ✓ Reduce load on leaf Prometheus servers.
- ✓ Centralized dashboards and alerts.

WHEN TO USE FEDERATION

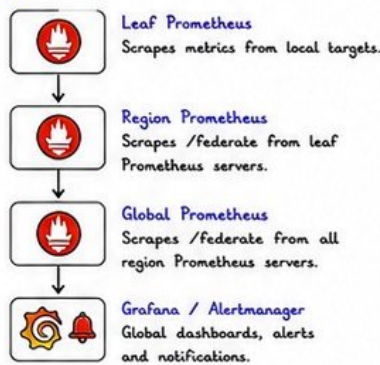
- 10k+ targets
- Multiple clusters / data centers.
- Multi-team / multi-tenant environments
- Long-term global visibility



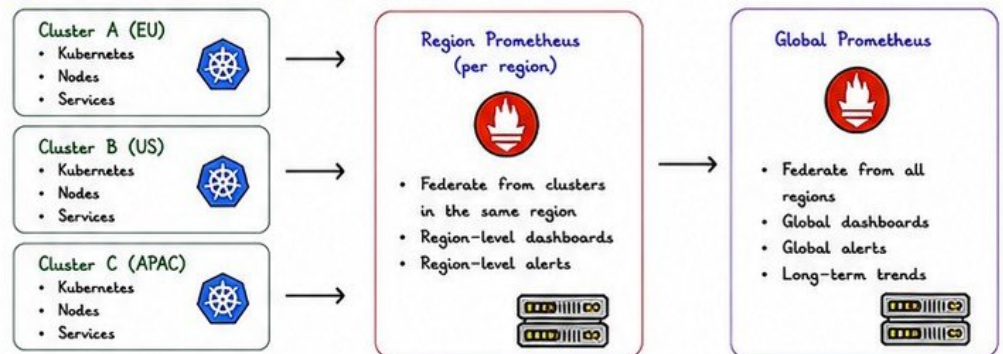
2 FEDERATION ARCHITECTURE (HIERARCHICAL)



3 GLOBAL MONITORING FLOW



4 MULTI-CLUSTER MONITORING EXAMPLE



5 ADVANTAGES

- ✓ Scales to millions of targets.
- ✓ Reduces cardinality at each level.
- ✓ Isolates failures (one cluster down doesn't break global view).
- ✓ Region / team autonomy.
- ✓ Better performance and reliability.
- ✓ Enables global dashboards and SLOs.



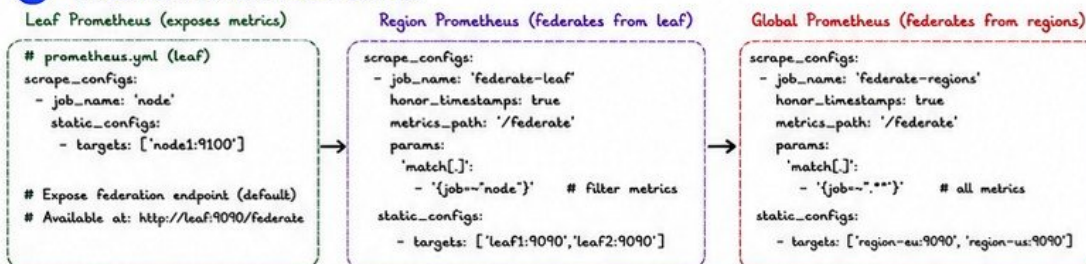
6 LIMITATIONS

- ✗ Data freshness depends on poll interval between levels.
- ✗ Not real-time across all layers.
- ✗ Complex to design and maintain.
- ✗ Label conflicts must be handled.
- ✗ Requires consistent metric naming.
- ✗ More moving parts = more monitoring to monitor.



Federation is powerful, but complexity grows with every layer.

7 FEDERATION CONFIGURATION EXAMPLE



8 PRO TIPS

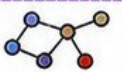
- ✓ Filter metrics using match[...] to reduce cardinality.
- ✓ Use meaningful labels like region, cluster, team.
- ✓ Set appropriate intervals (e.g., 30s - 1m).
- ✓ Monitor federation servers with alerts.
- ✓ Test federation with small metric sets first.



KEY TAKEAWAY



Federation = Scale + Structure + Global Visibility.
Design it early, keep it simple, and monitor the monitors.



15. REMOTE WRITE



KEY IDEA

Remote Write solves the problem of local storage limits by sending metrics to remote systems built for long-term storage & scale.



Remote Write allows Prometheus to send ("write") metrics to remote storage systems designed for **scale**, **cost efficiency**, and **long-term retention**.

1 WHAT IS REMOTE WRITE?

A Prometheus feature that continuously pushes samples to a remote endpoint via an HTTP API.



NOTES

- Asynchronous, non-blocking.
- Reliable with retry + queue + WAL.
- Works in parallel with local storage.

2 WHY USE REMOTE WRITE?

LONG-TERM STORAGE



Keep metrics for months or years beyond local retention.

INFINITE SCALE



Handle billions of time series and high-cardinality data.

COST EFFICIENCY



Object storage + compression = lower cost than local disks.

CENTRALIZED OBSERVABILITY



Aggregate metrics from many clusters, regions, or teams in one place.

SEPARATION OF CONCERNS



Prometheus focuses on collection; remote system handles storage at scale.

3 LONG-TERM STORAGE

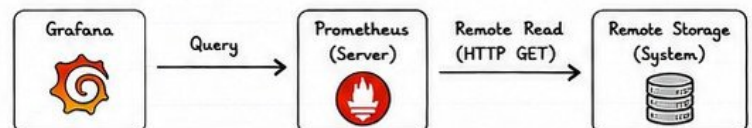
- Local Prometheus storage: short-term (days/weeks).
- Remote storage: optimized for long-term (months/years).
- Enables trend analysis, capacity planning, audits.

STORAGE	TYPICAL RETENTION	USE CASE
Local Prometheus	7d - 30d	Real-time monitoring, alerts, dashboards
Remote Storage	3m - 5y+	Historical analysis, compliance, ML, reports

★ TIP: Keep hot data local, cold data remote.

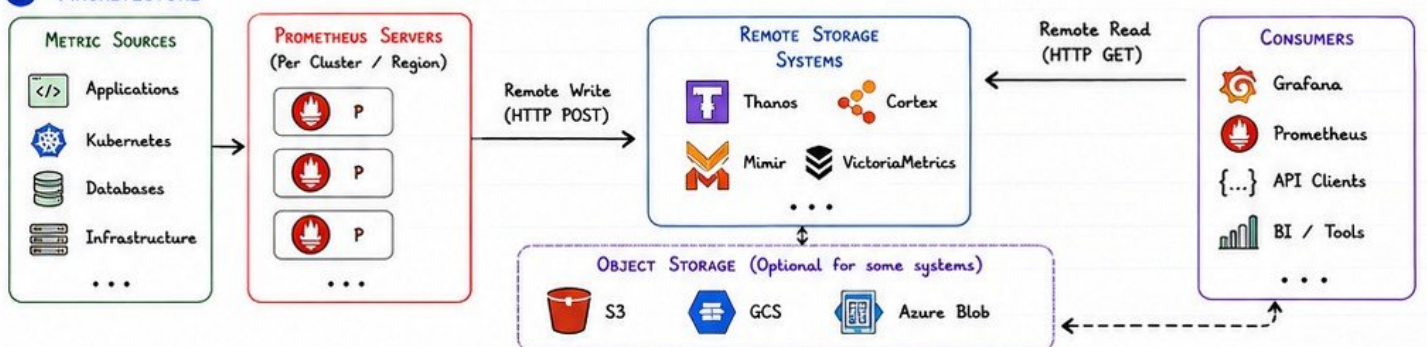
4 REMOTE READ

Remote Read allows Prometheus (or Grafana) to query data from remote storage as if it were local.



- Extends query horizon beyond local retention.
- Often used for dashboards needing long-term data.
- Can combine local + remote seamlessly.

5 ARCHITECTURE



6 PRODUCTION USE CASES



Multi-Cluster / Multi-Region
Aggregate metrics from all clusters to a single global view.



Long-Term Trend Analysis
Store 1+ year of metrics for capacity planning & reporting.



Cost Optimization
Use cheaper object storage for historical data.



Compliance & Audit
Keep metrics for regulatory needs (SOX, PCI-DSS, ISO, etc.).



ML & Anomaly Detection
Feed long-term metrics into ML models.



Disaster Recovery
Restore visibility even if a cluster or region is lost.

7 BEST PRACTICES

- ✓ Enable WAL for durability during outages.
- ✓ Use batching + compression (recommended by default).
- ✓ Set appropriate queue_config for throughput & retries.
- ✓ Monitor prometheus_remote_storage_*. metrics.
- ✓ Test failover: if remote down, Prometheus must not block.
- ✓ Use secure endpoints (HTTPS + auth).
- ✓ Keep retention policies aligned with storage lifecycle.

Example (prometheus.yml)

```
remote_write:
- url: https://thanos-receive.monitoring.svc:19291/api/v1/receive
  remote_timeout: 30s
  queue_config:
    max_samples_per_send: 10000
    max_shards: 200
    capacity: 25000
    max_retries: 10
    min_backoff: 30ms
    max_backoff: 5s
```

NOTE:

Tune queue_config based on your volume & bandwidth.

PRO TIPS



Keep 7-30 days local.



Cold data goes remote.



Use Remote Read for deep queries.

16. THANOS



KEY IDEA

Thanos extends Prometheus to solve long-term storage, global query, high availability and downsampling to Prometheus without losing its power and simplicity.



Thanos is a set of components that add long-term storage, global query, high availability and downsampling to Prometheus without losing its power and simplicity.

1 WHY THANOS?

- ☆ Prometheus was not designed for multi-year retention.
- ✓ Need global view across many clusters and regions.
- ✓ Reduce storage cost with object storage + downsampling.
- ✓ Improve availability (no single point of failure).
- ✓ Keep Prometheus simple and fast for short-term storage.



2 THANOS COMPONENTS OVERVIEW

SIDE CAR



Runs next to Prometheus. Uploads blocks to object storage and exposes them for querying.

STORE GATEWAY



Provides access to blocks stored in object storage. Does NOT store data locally. Stateless & scalable.

QUERIER



The brain. Queries data from Prometheus + Store Gateways and deduplicates results.

COMPACTOR



Compacts blocks, applies retention policies and performs downsampling.

OBJECT STORAGE



Durable, cheap, highly available storage for long-term metric blocks.

3 COMPONENTS DEEP DIVE

SIDE CAR

- Runs next a sidecar container with Prometheus.
- Watches local TSDB block directory.
- Uploads new blocks to object storage.
- Exposes gRPC API for Store Gateway & Querier.
- Supports HA deduplication.

Port: 10950 (gRPC)
Reads local blocks from --storage.tsdb.path

STORE GATEWAY

- Reads blocks from object storage on demand.
- Serves them to Querier.
- Stateless - can add more replicas anytime.
- Caches index & chunks in memory for speed.

Port: 10954 (gRPC)
No local persistent storage required.

QUERIER

- Accepts PromQL queries.
- Queries Prometheus + Store Gateways.
- Merges & deduplicates results.
- Enables global query across clusters & regions.

Port: 10900 (HTTP)
Reads real-time + long-term data.

COMPACTOR

- Groups small blocks into larger blocks.
- Enforces retention (e.g. keep 1y).
- Creates downsampled blocks (5m, 1h, 1d).
- Runs periodically.

Port: 10902 (gRPC)
Saves cost & improves query performance.

OBJECT STORAGE

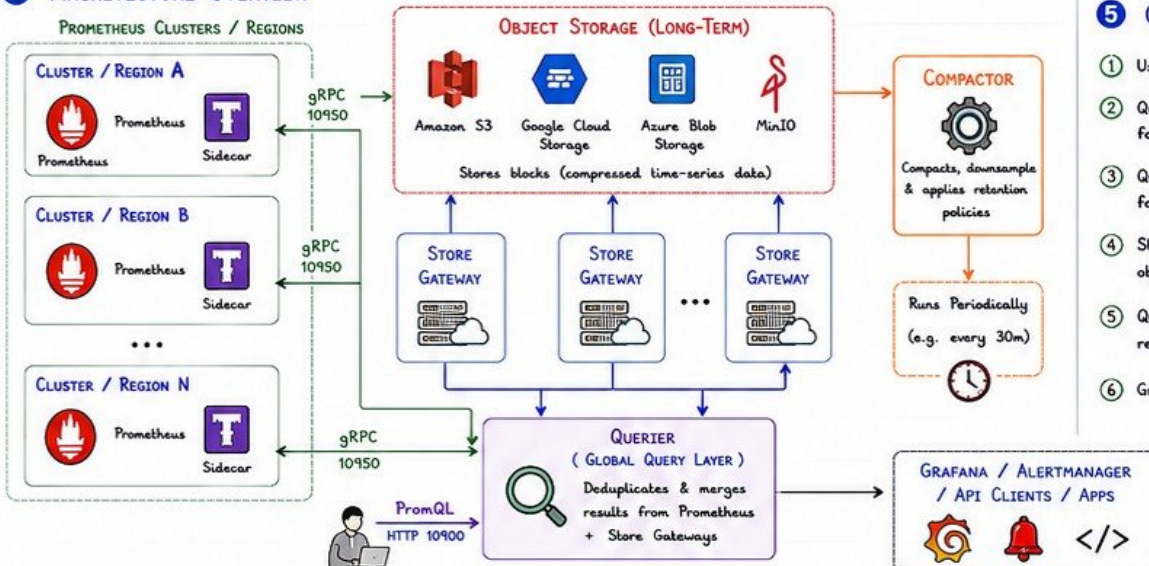
- Stores all uploaded blocks.
- Highly durable.
- Examples: S3, GCS, Azure Blob, MinIO.
- Least expensive place for long-term metrics.

Structure:
bucket/prefix/
tenant/ulid/block



4 ARCHITECTURE OVERVIEW

PROMETHEUS CLUSTERS / REGIONS



5 GLOBAL QUERY FLOW

- ① User sends query to Querier.
- ② Querier queries local Prometheus for recent data.
- ③ Querier queries Store Gateways for historical data.
- ④ Store Gateways read blocks from object storage and return data.
- ⑤ Querier merges, deduplicates and returns the result.
- ⑥ Grafana/Clients display the data.

6 BENEFITS

- ✓ Long-term retention at low cost
- ✓ Global view across clusters/regions
- ✓ High availability (no single Prometheus dependency)
- ✓ Downsampling for faster queries
- ✓ Elastic & horizontally scalable



7 COMMON USE CASES

- Multi-cluster / multi-region monitoring
- Compliance: keep metrics for 1+ year
- Reduce Prometheus server storage pressure
- Business dashboards spanning many environments
- Global SRE / NOC observability

8 TIPS

- ✓ Use dedicated bucket & prefix per environment.
- ✓ Enable bucket versioning + lifecycle policies.
- ✓ Set retention in Compactor (e.g. 1y raw, 5y downsampled).
- ✓ Run Store Gateway with sufficient memory (cache).
- ✓ Monitor Thanos components with Prometheus!

QUICK COMMANDS

Check Thanos components
kubectl -n monitoring get pods -l app=thanos

Query via Thanos Querier
curl "http://querier:10900/api/v1/query?query=up"

View buckets / blocks
aws s3 ls s3://my-metrics-bucket/ --recursive

17. CORTEX & GRAFANA MIMIR



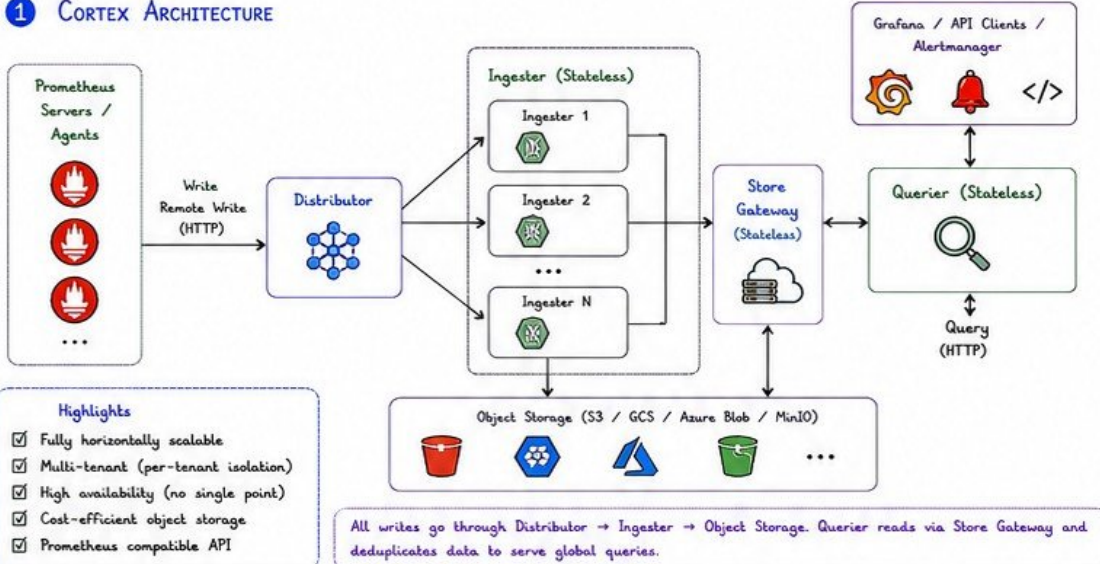
KEY IDEA

Cortex and Mimir are horizontally scalable, multi-tenant, long-term metric storage systems built for Prometheus at massive scale. Thanos is great for simplicity & flexibility. Cortex/Mimir is built for multi-tenancy & scale.

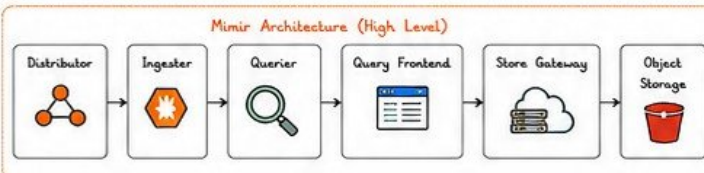


Cortex (by Weaveworks, now Grafana Labs) and Grafana Mimir (open source, CNCF) are Prometheus-compatible metric backends designed for **massive scale**, **multi-tenancy**, **high availability** and **long-term storage**.

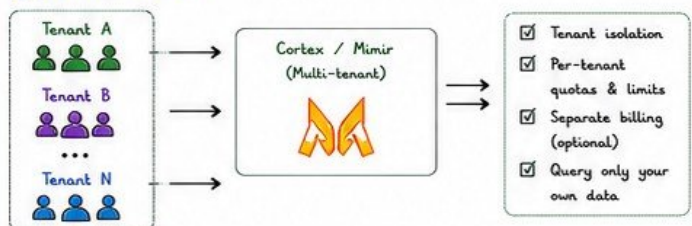
1 CORTEX ARCHITECTURE



2 GRAFANA MIMIR OVERVIEW



3 MULTI-TENANCY



4 SCALABILITY

- ✓ Add more Ingestor nodes → more write throughput
- ✓ Add more Store Gateways → more read throughput
- ✓ Stateless components → easy horizontal scaling
- ✓ Object storage → unlimited, cheap, durable
- ✓ Handle billions of time series & high cardinality



5 COMPARISON WITH THANOS

Thanos	Cortex / Mimir
✓ Best for single-tenant setups	✓ Built for multi-tenancy
✓ Flexible & modular	✓ Massive scale (10K+ tenants)
✓ Uses object storage	✓ High cardinality optimized
✓ Global query via components	✓ Built-in HA & scalability
✓ Less ops overhead for small-medium scale	✓ More components (higher ops)
✓ Not built-in multi-tenancy	✓ Better cost efficiency at scale

6 WHEN TO CHOOSE EACH?

Choose Thanos if:	Choose Cortex / Mimir if:
✓ You run a small/medium environment. ✓ You want simplicity & modularity. ✓ You have few teams / no multi-tenancy. ✓ You want to use existing object storage. ✓ You like open source with minimal components.	✓ You need multi-tenancy (many teams/customers). ✓ You have massive scale & high cardinality. ✓ You need long-term retention (months/years). ✓ You want built-in scalability & HA. ✓ You operate a monitoring SaaS or large platform.

7 COMPARISON TABLE

FEATURE	THANOS	CORTEX	GRAFANA MIMIR
Multi-Tenancy	X Limited (external)	✓ Built-in	✓ Built-in
Scalability	Medium	Very High	Very High
Components	5 core components	10+ components	10+ components
High Cardinality	Good	Excellent	Excellent
Long-Term Storage	Yes (Object Storage)	Yes (Object Storage)	Yes (Object Storage)
HA	Good	Excellent	Excellent
Operational Complexity	Lower	Higher	Higher
Best For	Single-tenant, simplicity	Large scale, SaaS	Large scale, SaaS (OSS)
Project	CNCF	CNCF (Incubating)	CNCF (Graduated)

PRO TIPS



Use recording rules at edge (near Prometheus) to reduce cardinality & cost.



Store raw metrics short-term locally, aggregated metrics long-term remotely.



Set per-tenant quotas (limit series, ingestion rate, retention).



Monitor your monitoring system (Cortex/Mimir components too!).

18. HIGH AVAILABILITY



KEY IDEA

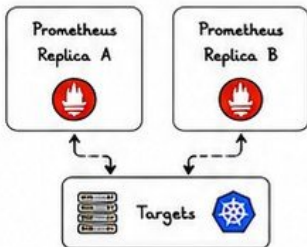
High Availability ensures monitoring never goes down when components fail. Design for redundancy, failover and data durability.



A highly available Prometheus stack guarantees monitoring, alerting and data are available even during failures, restarts or maintenance.

1 HA PROMETHEUS

- Run multiple Prometheus replicas.
- Each replica scrapes targets independently.
- Replicas are "active-active".
- No single point of failure.
- Store metrics in shared remote storage (Thanos/Cortex/Mimir) for durability.



2 MULTIPLE REPLICAS

- 2 or more replicas across failure domains (AZ/Region).
- Independent resources (CPU, RAM, Disk).
- Use distinct external labels to identify replicas.
- Example external labels:

```
prometheus_replica: A
prometheus_replica: B
```

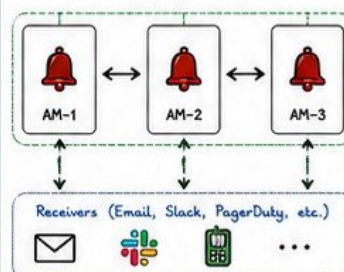
External Label Benefits

- Deduplication
- HA querying
- Troubleshooting

3 ALERTMANAGER HA

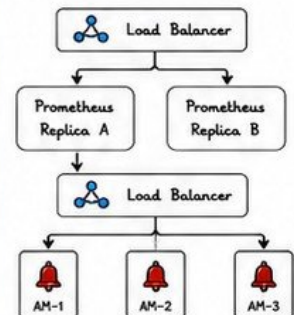
- Run multiple Alertmanager instances.
- They form a cluster using Gossip protocol.
- Alerts are deduplicated and highly available.

Alertmanager Cluster



4 LOAD BALANCING

- Use a Load Balancer in front of Prometheus and Alertmanager.
- Distributes traffic across replicas.
- Health checks ensure only healthy instances receive traffic.



5 FAILOVER

- If a Prometheus replica fails, others continue scraping.
- If an Alertmanager node fails, others take over automatically.
- Remote storage ensures no data loss.
- DNS/LB automatically routes traffic to healthy instances.

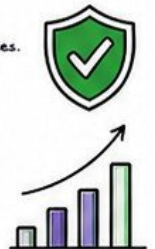


6 STORAGE CONSIDERATIONS

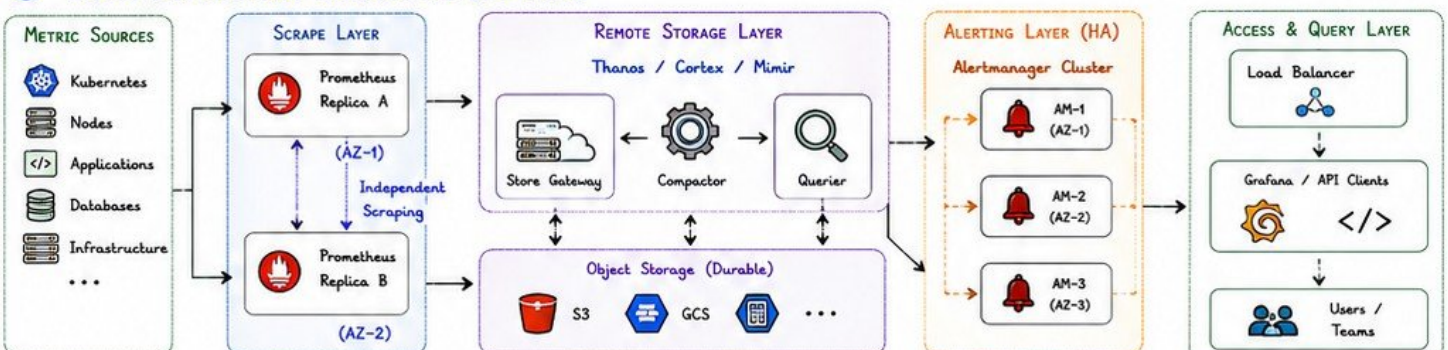
- Local storage is not enough for HA.
- Use remote storage for long-term retention and durability.
- Options:
 - Thanos (Object Storage + Compactor)
 - Cortex / Grafana Mimir
 - Remote Write to long-term store

7 BEST PRACTICES

- Deploy across multiple AZs.
- Use anti-affinity rules in Kubernetes.
- Enable persistent volume for WAL.
- Use remote_write for durability.
- Monitor the monitoring stack.
- Test failover regularly.
- Keep components updated.



8 PRODUCTION REFERENCE ARCHITECTURE (HA STACK)



Monitoring
(Node Exporter)



Logging



Alerting
(on stack health)



Dashboards



Backups



Runbooks &
Automation

PRO TIPS



Replicas + Remote Storage
= High Availability
+ Long-term Durability



Always test failure
scenarios before
production.



Retention policies
should match your
business needs.



Keep an eye on WAL
size, disk usage
and compactor health.



Monitoring the
monitoring stack
is critical!

19. TROUBLESHOOTING PRODUCTION INCIDENTS

KEY IDEA
Follow a methodical workflow and know where to look. Understand the symptoms, find the root cause, and prevent it from happening again.

Incidents happen. The key is to detect fast, debug smart, and restore quickly.

1 REAL-WORLD SCENARIOS

1 MISSING METRICS



Symptoms

- Graphs flatline
- No new data

What to check

- Target down?
- Exporter running?
- Network issues?
- relabel_configs?

2 TARGET DOWN



Symptoms

- Target shows DOWN
- Scrape failing

What to check

- Target URL accessible?
- Firewall / SG rules?
- Service/Pod running?
- DNS resolution?

3 HIGH CARDINALITY



Symptoms

- High memory usage
- Slow queries
- Many series

What to check

- Unbounded labels?
- User IDs, IPs, UUIDs?
- Use relabeling/drop rules

4 SLOW PROMQL QUERIES



Symptoms

- Queries take too long
- Timeout errors

What to check

- High cardinality?
- Large time range?
- Missing recording rules?
- Inefficient functions?

5 TSDB CORRUPTION



Symptoms

- Prometheus won't start
- Errors in logs
- Data missing

What to check

- Check TSDB logs
- Run tsdb tool verify
- Restore from backup

6 ALERT STORMS



- Too many alerts
- Channels flooded
- Restored from backup

What to check

- Alert rules firing too often?
- Missing for: duration?
- Lack of inhibition rules?
- Check alertmanager logs

7 SCRAPE FAILURES



Symptoms

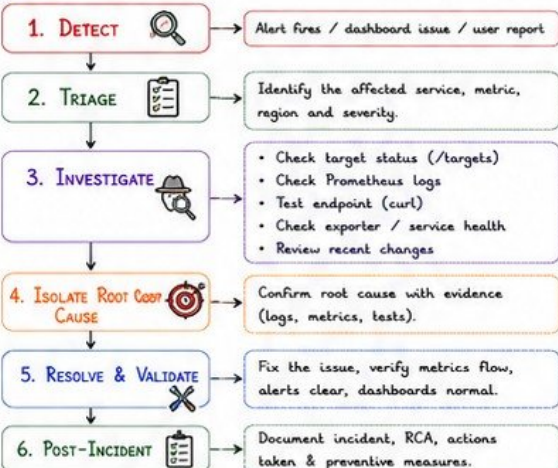
- Endpoint slow/unhealthy?
- Timeouts too low?
- Network latency?

- Scrape duration high
- Errors in scrape
- Target flapping

What to check

- Version compatibility
- Config & permissions?

2 DEBUGGING WORKFLOW



3 QUICK CHECKLIST

- ☒ Target UP in /targets?
- ☒ Endpoint reachable (curl)?
- ☒ Exporter healthy?
- ☒ Prometheus logs clean?
- ☒ Scrape duration normal?
- ☒ Recent config changes?
- ☒ Resource usage OK (CPU / RAM / Disk)?
- ☒ Alertmanager healthy?
- ☒ Disk space available?
- ☒ Any cardinality explosion?

4 USEFUL COMMANDS & TOOLS

- Check targets status
`curl -s http://localhost:9090/api/v1/targets`
- Check Prometheus config
`promtool check config prometheus.yml`
- Reload Prometheus
`curl -X POST http://localhost:9090/-/reload`
- Query instant value
`curl -G http://localhost:9090/api/v1/query \`
`--data-urlencode "query=up"`
- TSDB verify
`promtool tsdb verify /prometheus`
- TSDB analyze
`promtool tsdb analyze /prometheus`
- Check TSDB head stats
`curl -s http://localhost:9090/api/v1/status/tsdb`

5 COMMON LOG PATTERNS

LOG PATTERN	MEANING	WHAT TO DO
context deadline exceeded	Scrape timeout	Increase timeout / fix endpoint
connection refused	Endpoint not reachable	Check service / network
no such host	DNS issue	Check DNS / service discovery
out of order sample	Timestamps out of order	Check clock / exporter
storage: corrupt head block	TSDB corruption	Run tsdb verify / restore backup
sample rejected: too old	Too old samples	Check clock skew / exporter

6 PREVENTIVE TIPS

- ☒ Use recording rules to optimize queries.
- ☒ Control cardinality with relabeling.
- ☒ Set appropriate timeouts & limits.
- ☒ Monitor Prometheus itself.
- ☒ Keep backups of TSDB.
- ☒ Test alerts and dashboards regularly.
- ☒ Review changes with change control.

Remember:

Good monitoring is measured not when everything is working, but when something breaks.



PRO TIPS



Use staging to test rule changes before prod.



Use inhibition & silences wisely.



Keep dashboards simple, clear & actionable.



Back up Prometheus regularly.



Run game days to test your setup.

20. PRODUCTION MONITORING PROJECT



KEY IDEA

Everything you've learned, put into practice! Build, test, and operate a complete monitoring system like in real production environments.



HANDS-ON PROJECT: Build a production-ready monitoring stack that **collects**, **stores**, **visualizes** and **alerts** on real infrastructure and application metrics.

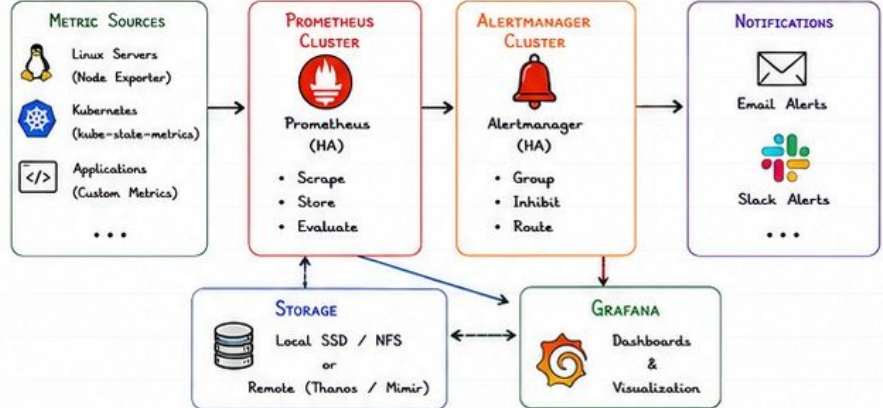
1 PROJECT GOAL

Design and deploy a production-ready monitoring stack that:

- ✓ Collects metrics from servers, apps & K8s
- ✓ Stores metrics for long-term analysis
- ✓ Visualizes with beautiful dashboards
- ✓ Sends alerts via Email and Slack
- ✓ Is highly available and reliable



2 PRODUCTION ARCHITECTURE



3 STACK COMPONENTS

	PROMETHEUS	Metrics collection, storage & alerting rules
	ALERTMANAGER	Alert routing, grouping, inhibition, silencing
	GRAFANA	Dashboards, visualization & data exploration
	NODE EXPORTER	System level metrics (CPU, Memory, Disk, etc.)
	KUBE-STATE-METRICS	Kubernetes object metrics (Pods, Deploy, etc.)

4 PROMETHEUS CONFIG (prometheus.yml)

```
global:
  scrape_interval: 15s
scrape_configs:
  - job_name: 'node_exporter'
    static_configs:
      - targets: ['node1:9100', 'node2:9100']
  - job_name: 'kubernetes'
    kubernetes_sd_configs:
      - role: endpoints
    relabel_configs:
      - action: keep
        source_labels: [__meta_kubernetes_service_label_app]
        regex: kube-prometheus-stack-prometheus
  - job_name: 'kubernetes-metrics'
    static_configs:
      - targets: ['kubernetes-metrics.kube-system.svc:8080']
```

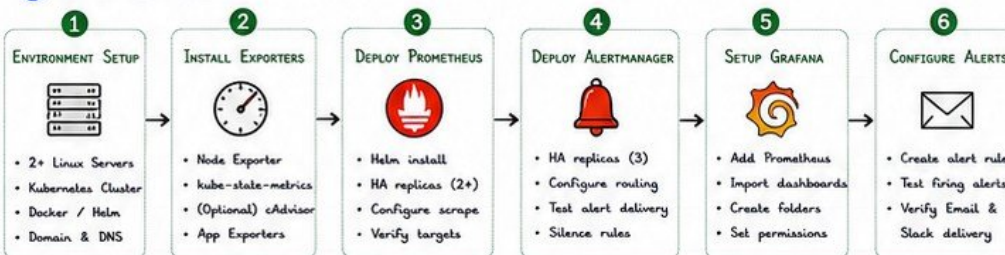
5 ALERT RULE EXAMPLE (alert-rules.yml)

```
groups:
  - name: example-alerts
    rules:
      - alert: HighCPUUsage
        expr: 100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High CPU usage on {{ $labels.instance }}"
          description: "CPU usage is above 80% for more than 5 minutes."
```

6 ALERT ROUTING (alertmanager.yml)

```
route:
  group_by: ['alertname']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 3h
  receiver: 'team-email'
  routes:
    - match:
        severity: critical
      receiver: 'team-slack'
receivers:
  - name: 'team-email'
    email_configs:
      - to: 'ops-team@example.com'
        from: 'alerts@example.com'
        smarthost: 'smtp.gmail.com:587'
        auth_username: 'alerts@example.com'
        auth_password: '*****'
  - name: 'team-slack'
    slack_configs:
      - api_url: 'https://hooks.slack.com/services/XXXXX'
        channel: '#alerts'
        send_resolved: true
```

7 DEPLOYMENT STEPS



8 PROJECT DELIVERABLES

- ✓ Working monitoring stack (HA)
- ✓ System & K8s dashboards
- ✓ Alert rules (CPU, Memory, Disk, Down)
- ✓ Email alerts configured
- ✓ Slack alerts configured
- ✓ Documentation (setup & runbook)
- ✓ GitHub repo with configs
- ✓ Bonus: Long-term storage with Thanos/Mimir

IDEAS FOR ALERT RULES

- Instance DOWN
- High CPU / Memory
- Disk Space Low
- High Load Average
- Pod CrashLoop
- K8s Node Not Ready



TEST & VERIFY

- ✓ All targets UP
- ✓ Dashboards showing data
- ✓ Alerts firing
- ✓ Email received
- ✓ Slack notification received
- ✓ Failover test (kill a pod / node)

★ **SUCCESS CRITERIA:** Your monitoring stack is reliable, observable, and alerts you before users are affected!

PRO TIPS



Start simple and iterate. Add alerts gradually.



Keep dashboards focused and easy to read.



Test alerts regularly. Broken alerts are worse than none.



Plan for long-term storage from day one.



Document everything. Your future self will thank you!



Monitor your monitoring!

Shared by convenient and free Image to PDF
Converter: <https://st.simplesign.itd/7ziEng>